

企业内部智能知识助手项目介绍文档

1. 项目名称

企业内部智能知识助手系统

Enterprise Internal AI Knowledge Assistant

2. 项目背景

在企业日常办公中，员工经常需要查询大量内部资料，例如报销流程、请假制度、合同模板、部门规范、系统操作说明等。这些信息通常分散在 PDF、Word、Wiki、OA 系统或企业内部数据库中，查找效率较低。

随着大语言模型和 AI Agent 技术的发展，可以通过构建一个面向企业内部场景的智能助手，让员工通过自然语言提问，快速获得准确答案，并在需要时自动调用企业系统接口完成部分业务操作。

本项目希望模拟真实企业中的 AI 落地场景，构建一个具备 **RAG 知识问答**、**Agent 工具调用**、**权限控制**、**多轮对话** 能力的企业级智能助手系统。

3. 项目目标

本项目的核心目标是：

1. 构建一个可以基于企业内部文档进行智能问答的系统。
 2. 实现 AI Agent 对企业业务工具的调用能力。
 3. 通过 Java 后端管理用户、权限、业务接口和系统数据。
 4. 通过 Python 服务实现 RAG、LLM 调用、Agent 编排等 AI 能力。
 5. 模拟企业真实业务场景，形成一个适合实习求职展示的完整项目。
-

4. 适用业务场景

该项目可以覆盖企业中常见的办公和业务支持场景：

4.1 企业制度问答

员工可以询问：

- “公司的报销流程是什么？”
- “试用期员工可以请年假吗？”
- “出差补贴标准是多少？”
- “合同审批需要哪些步骤？”

系统会从企业制度文档、流程文件或知识库中检索相关内容，并结合大语言模型生成答案。

4.2 内部资料检索

员工可以通过自然语言查找：

- 合同模板
- 报销说明
- 入职手册
- 项目规范
- 产品说明文档

系统通过向量检索找到相关文档片段，并返回整理后的答案和来源。

4.3 业务状态查询

Agent 可以调用企业内部接口，例如：

- 查询报销审批进度
- 查询请假剩余额度
- 查询员工个人信息
- 查询项目任务状态

例如用户问：

“帮我查一下我这个月的报销进度。”

系统判断这是一个业务查询请求，然后调用后端提供的报销查询接口，返回结构化结果。

4.4 文案和邮件生成

系统可以根据用户需求生成企业办公文本，例如：

- 请假申请邮件
- 报销说明
- 项目周报
- 会议纪要
- 客户沟通邮件

5. 核心功能设计

5.1 用户登录与权限管理

功能说明：

- 用户注册和登录
- JWT 鉴权
- 角色区分：普通员工、管理员
- 不同角色访问不同数据
- 用户只能查询自己相关的业务信息

企业价值：

权限控制是企业系统中非常重要的一部分。这个功能可以体现项目不是简单的 AI Demo，而是具备企业级系统设计意识。

5.2 文档上传与知识库构建

功能说明：

- 管理员上传企业文档
- 支持 PDF、Word、Markdown、TXT 等格式
- 文档解析后进行分块
- 对文本块生成 Embedding
- 存入向量数据库

处理流程：

上传文档 → 文档解析 → 文本切分 → 生成向量 → 存入向量数据库

5.3 RAG 智能问答

功能说明：

- 用户输入问题
- 系统将问题转成向量
- 从向量数据库检索相关文档片段
- 将检索结果和用户问题一起传给大语言模型
- 生成最终回答

处理流程：

用户问题 → 向量检索 → 召回相关文档 → 构造 Prompt → LLM 生成回答

项目亮点：

- 不是简单调用大模型，而是结合企业私有知识库回答问题。
 - 可以降低模型胡编乱造的概率。
 - 可以返回答案来源，提高可信度。
-

5.4 Agent 工具调用

功能说明：

系统不只回答问题，还可以调用工具完成任务。

可实现的工具包括：

1. 查询员工信息工具
2. 查询报销进度工具
3. 查询请假余额工具
4. 生成邮件工具
5. 查询知识库工具

示例：

用户输入：

“帮我查一下我的年假还剩几天。”

Agent 判断该问题需要调用请假系统接口，然后执行：

识别意图 → 选择工具 → 调用 Java 后端接口 → 获取结果 → 生成自然语言回复

5.5 多轮对话与上下文记忆

功能说明：

系统可以保存用户最近几轮对话，支持上下文理解。

示例：

用户：

“公司的报销流程是什么？”

系统回答后，用户继续问：

“那发票有什么要求？”

系统能够理解“发票要求”仍然和报销流程相关。

5.6 日志与调用记录

功能说明：

- 记录用户提问
- 记录模型调用次数
- 记录工具调用情况
- 记录错误日志
- 记录 Token 使用量

企业价值：

日志可以用于问题排查、成本统计、安全审计和系统优化。

6. 系统架构设计

推荐采用 Java + Python 混合架构。



7. 技术栈选择

7.1 后端技术

技术	用途
Java	企业后端主语言
Spring Boot	构建后端服务
Spring Security / JWT	登录认证和权限控制
MySQL	存储用户、文档、业务数据
Redis	缓存、会话、限流，可选
MyBatis / JPA	数据库访问
RESTful API	Java 与 Python 服务通信

7.2 AI 服务技术

技术	用途
Python	AI Agent 服务开发
FastAPI	提供 AI 服务接口
LangChain / LangGraph	Agent 编排和工具调用
FAISS / Milvus	向量检索
OpenAI / 通义千问 / 智谱 AI	大语言模型调用
Embedding Model	文本向量化

建议：

- 初学阶段可以使用 **FAISS**，部署简单。
- 想体现企业级能力，可以使用 **Milvus**。
- Agent 编排建议使用 **LangGraph**，比普通 LangChain Agent 更适合展示复杂流程。

7.3 前端技术

前端不需要做得太复杂，重点展示功能即可。

可选技术：

技术	用途
Vue / React	构建页面
Axios	调用后端接口
Element Plus / Ant Design	快速搭建管理后台

核心页面：

1. 登录页面
2. 聊天问答页面
3. 文档上传页面
4. 管理员后台页面
5. 调用日志页面

8. 数据库表设计建议

8.1 用户表 user

字段	说明
id	用户 ID

字段	说明
username	用户名
password	密码
role	角色
department	部门
create_time	创建时间

8.2 文档表 document

字段	说明
id	文档 ID
file_name	文件名
file_type	文件类型
uploader_id	上传者
status	处理状态
create_time	上传时间

8.3 对话记录表 chat_record

字段	说明
id	记录 ID
user_id	用户 ID
question	用户问题
answer	系统回答
source_docs	引用文档
create_time	创建时间

8.4 工具调用记录表 tool_call_log

字段	说明
id	记录 ID
user_id	用户 ID
tool_name	工具名称
input_params	输入参数
output_result	输出结果

字段	说明
status	调用状态
create_time	调用时间

9. 接口设计示例

9.1 用户登录接口

POST /api/auth/login

请求参数：

```
{
  "username": "employee01",
  "password": "123456"
}
```

返回结果：

```
{
  "token": "jwt_token",
  "role": "employee"
}
```

9.2 智能问答接口

POST /api/chat/ask

请求参数：

```
{
  "question": "公司的报销流程是什么？"
}
```

处理流程：

Java 接收请求 → 校验用户身份 → 调用 Python AI 服务 → 保存对话记录 → 返回答案

9.3 文档上传接口

`POST /api/document/upload`

处理流程：

上传文件 → 保存文件信息 → 调用 Python 文档处理接口 → 构建向量索引

9.4 Agent 工具调用接口

`POST /api/agent/execute`

示例请求：

```
{
  "message": "帮我查一下我的报销进度"
}
```

系统会自动判断是否需要调用工具。

10. 落地实施步骤

阶段一：完成基础后端系统

目标：先搭好企业系统底座。

需要完成：

1. 创建 Spring Boot 项目
2. 配置 MySQL 数据库
3. 实现用户登录和 JWT 鉴权
4. 实现用户角色权限控制
5. 实现基础业务接口，例如员工信息查询、报销进度查询

产出：

- 可运行的 Java 后端服务
- 基础数据库表
- 登录和权限功能

阶段二：完成 Python AI 服务

目标：实现 AI 问答和 Agent 能力。

需要完成：

1. 创建 FastAPI 项目
2. 接入大语言模型 API
3. 实现普通对话接口
4. 实现 Prompt 模板
5. 实现基础 Agent 工具调用
6. 提供接口给 Java 后端调用

产出：

- Python AI 服务
 - /chat 接口
 - /agent/execute 接口
-

阶段三：实现文档知识库和 RAG

目标：让系统能够基于企业文档回答问题。

需要完成：

1. 实现文档上传
2. 实现文档解析
3. 实现文本切分
4. 生成 Embedding
5. 存入 FAISS 或 Milvus
6. 实现向量检索
7. 将检索内容拼接进 Prompt

产出：

- 企业知识库
 - RAG 问答功能
 - 支持答案来源展示
-

阶段四：实现 Agent 工具调用

目标：让系统不只会回答，还能调用业务接口。

建议先实现 3 个工具：

1. 查询员工信息
2. 查询报销进度
3. 查询请假余额

工具调用流程：

用户提问 → Agent 判断意图 → 选择工具 → 调用 Java API → 整理结果 → 返回用户

产出：

- Agent 工具注册机制
- 业务接口调用能力
- 工具调用日志

阶段五：完善前端页面

目标：让项目更完整、更适合演示。

需要完成：

1. 登录页面
2. 聊天页面
3. 文档上传页面
4. 管理员文档管理页面
5. 日志查看页面

产出：

- 可演示的完整系统
- 面试时可以直接展示操作流程

阶段六：项目优化

可以加入以下优化点：

1. Redis 缓存常见问题答案
2. 对模型调用做限流
3. 对用户问题做敏感信息过滤
4. 增加答案来源引用
5. 增加调用失败重试机制
6. 记录 Token 消耗
7. 增加 Docker Compose 一键启动

11. 最小可行版本 MVP

如果时间有限，可以先完成以下功能：

1. Java 后端登录系统
2. Python FastAPI AI 问答服务
3. 文档上传和 RAG 问答

4. 一个业务工具调用，例如查询报销进度
5. 简单前端聊天页面

MVP 版本已经足够用于实习简历和面试展示。

12. 推荐开发顺序

建议按下面顺序开发：

1. Java 登录和权限
2. Java 模拟业务接口
3. Python 普通 LLM 问答
4. Java 调用 Python AI 服务
5. 文档上传和解析
6. 向量库和 RAG
7. Agent 工具调用
8. 前端页面
9. 日志、监控、优化
10. Docker 部署

不要一开始就做复杂 Agent。先保证系统能跑通，再逐步增强 AI 能力。

13. 项目难点与解决方案

13.1 模型容易胡编乱造

解决方案：

- 使用 RAG 检索企业文档
- 在 Prompt 中要求模型只能基于资料回答
- 返回答案来源
- 当知识库没有答案时，明确回答“未找到相关资料”

13.2 Agent 工具调用不稳定

解决方案：

- 限定工具范围
- 对工具参数做校验
- 记录工具调用日志
- 工具调用失败时返回明确错误

13.3 企业数据权限问题

解决方案：

- Java 后端负责权限校验

- Python Agent 不直接访问数据库
- 所有业务数据都通过 Java API 获取
- 用户只能访问自己有权限的数据

13.4 系统复杂度较高

解决方案：

- 先做 MVP
 - Java 和 Python 服务分开开发
 - 每个模块单独测试
 - 最后再集成
-

14. 项目可以写进简历的描述

简历项目名称

企业内部智能知识助手系统（Java + Python + AI Agent + RAG）

简历描述示例

基于 Spring Boot、FastAPI 和 LangGraph 构建企业内部智能知识助手系统，实现企业文档知识问答、Agent 工具调用、用户权限控制和对话日志管理。系统支持文档上传、文本切分、Embedding 向量化和语义检索，并结合大语言模型生成基于企业知识库的回答。同时设计了报销进度查询、请假余额查询等业务工具，Agent 可根据用户意图自动选择工具并调用 Java 后端接口完成业务查询。

简历亮点

- 设计 Java + Python 分层架构，Java 负责企业业务系统，Python 负责 AI Agent 能力。
 - 实现基于 RAG 的企业知识库问答，支持文档解析、向量检索和答案来源展示。
 - 实现 Agent 工具调用机制，可自动调用企业业务接口完成报销、请假等查询。
 - 通过 JWT 和角色权限控制保障企业数据访问安全。
 - 记录模型调用、工具调用和用户对话日志，支持后续审计和问题排查。
-

15. 面试讲解思路

面试时可以按以下顺序介绍：

1. 先说明项目背景：企业内部资料分散，员工查询效率低。
 2. 再说明项目目标：做一个能查知识、能调用工具的智能助手。
 3. 介绍系统架构：Java 管业务，Python 管 AI。
 4. 重点讲 RAG：文档解析、切分、向量化、检索、生成答案。
 5. 重点讲 Agent：根据用户意图选择工具，并调用企业 API。
 6. 强调权限控制：AI 不能绕过业务系统权限。
 7. 最后讲优化点：日志、缓存、限流、Docker 部署。
-

16. 项目总结

本项目不是一个简单的聊天机器人，而是一个模拟企业真实 AI 落地场景的智能业务助手。它结合了企业后端开发、权限控制、知识库检索、AI Agent、工具调用和系统集成等能力，非常适合用于实习求职展示。

项目最大的亮点在于：

使用 Java 构建稳定的企业业务系统，用 Python 构建灵活的 AI Agent 能力层，实现企业知识问答和业务自动化处理。

通过该项目，可以向面试官展示自己不仅会调用大模型 API，还理解 AI 在企业系统中的真实落地方式。